

INCIDENT RESPONSE PLAYBOOK



PRODUCTION INCIDENT RESPONSE OPERATING MODEL

WHAT QUALIFIES AS AN INCIDENT?



- Unplanned disruption or degradation
- Affects users, customers or critical services
- Violates SLOs or business objectives
- Requires coordinated response
- May involve availability, performance, data integrity or security

INCIDENT vs ALERT vs PROBLEM

TYPE	MEANING	EXAMPLE
INCIDENT	Actual or imminent impact to service or customers	Checkout API failing for users
ALERT	Signal that something may be wrong	CPU above 80% for 5 minutes
PROBLEM	Underlying cause of one or more incidents	Memory leak in application

INCIDENT LIFECYCLE



SEVERITY vs URGENCY

CONCEPT	MEANING	EXAMPLE
SEVERITY	How much impact on customers and the business	SEV-1: Service unavailable for majority of users
URGENCY	How quickly it requires action	Act now to limit customer impact

CUSTOMER AND BUSINESS IMPACT

-  Customer impact – number of users affected
-  Business impact – revenue, conversions, SLAs
-  Scope – single service or multiple services
-  Duration – brief or prolonged
-  Data impact – risk of data loss or corruption
-  Operational impact – dependent teams and systems

WHY RESTORATION PRECEDES ROOT-CAUSE ANALYSIS

- Stops ongoing customer and business impact
- Limits further damage and data loss
- Creates a stable environment for accurate analysis
- Prevents distraction and analysis under pressure



SENIOR ENGINEER INSIGHT

Optimize first for limiting harm and restoring service. Deep root-cause analysis comes after the system is stable.



@veriqta



@veriqta



@veriqta



@veriqta



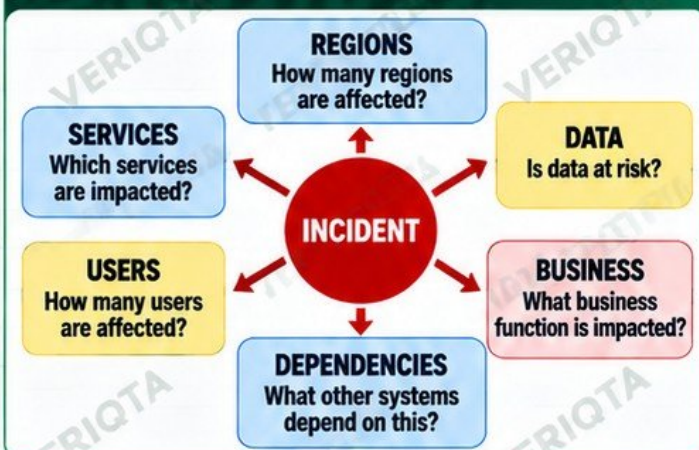
@veriqta



@veriqta

SEVERITY	CUSTOMER IMPACT	SCOPE	DURATION	DATA RISK	FINANCIAL RISK	EXAMPLES
SEV-1	Major outage many users cannot use service	Multiple regions or majority of users	Ongoing	High	Very high	Complete service outage, critical feature down, major dependency failure
SEV-2	Significant impact reduced functionality	Single region or large segment of users	Extended (> 30 minutes)	Medium	High	High error rate, major feature unavailable
SEV-3	Limited impact non-critical features	Small segment of users	Short (< 30 minutes)	Low	Medium	Degraded performance, intermittent errors
SEV-4	Minimal impact cosmetic or minor issue	Very small segment or internal only	Short	None	Low	UI issue, minor bug, non-critical service

BLAST-RADIUS ASSESSMENT



AVOID SEVERITY DEBATES

-  Do not spend time debating severity during an active incident.
-  Use impact and evidence, not opinions.
-  You can adjust severity as the situation becomes clearer.

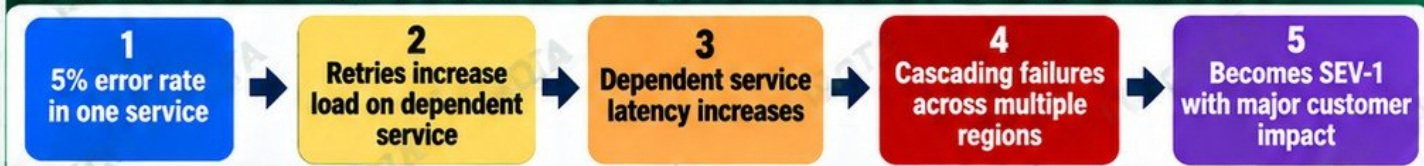
ESCALATION TRIGGERS

-  Customer impact is worsening
-  Multiple services or regions are affected
-  SLO or error budget is being burned quickly
-  Data loss or corruption is possible
-  Incident exceeds initial severity assumptions
-  Requires executive communication
-  Response team needs additional resources

UPGRADES AND DOWNGRADES

-  **Upgrade when:**
 - Wider impact than initially understood
 - Customer impact is increasing
 - Recovery is taking longer than expected
-  **Downgrade when:**
 - Impact is smaller than initially estimated
 - Service is stable and recovering
 - Customer impact is minimal

SCENARIO: SMALL ERROR RATE BECOMES A MULTI-REGION DEPENDENCY FAILURE



WHAT IS THE INCIDENT COMMAND SYSTEM?

- A structured way to manage incidents.
- Defines clear roles and responsibilities.
- Improves decision-making and communication.
- Ensures faster and safer recovery.
- Reduces confusion and duplicate work.

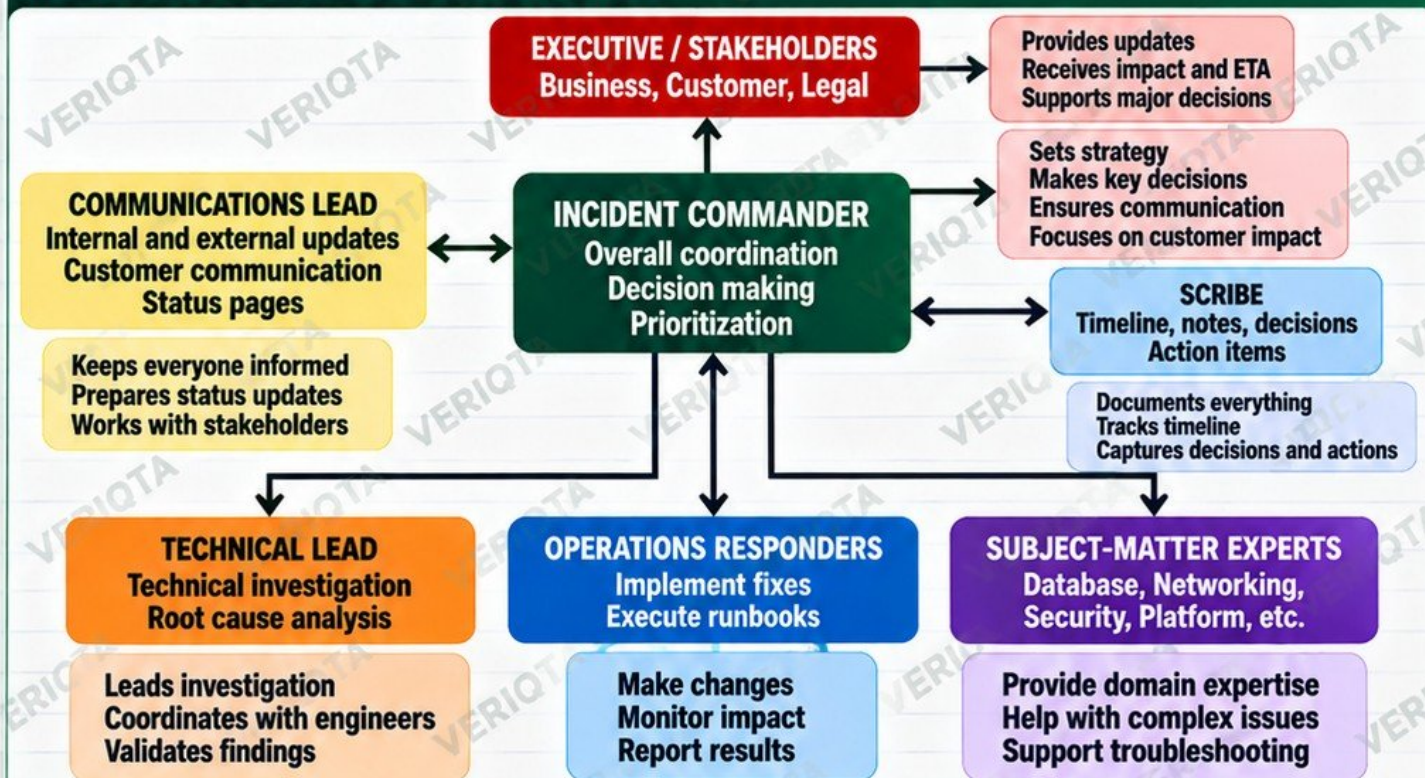
KEY ROLES

- 1 Incident Commander (Overall coordination)
- 2 Technical Lead (Technical investigation)
- 3 Operations Responders (Execute and implement fixes)
- 4 Communications Lead (Internal and external updates)
- 5 Scribe (Documentation and timeline)
- 6 Subject-Matter Experts (Domain-specific support)
- 7 Executive / Stakeholders (Business and customer interface)

WHY ROLES MATTER

- One person cannot coordinate, debug, communicate and document at the same time.
- Clear roles reduce chaos.
- Faster decisions.
- Better communication.
- Accurate documentation.
- Allows parallel work.
- Maintains focus on restoration and customer impact.

INCIDENT COMMAND STRUCTURE



DECISION OWNERSHIP

- Incident Commander owns overall decisions.
- Technical Lead recommends technical actions.
- Major changes are approved by the Incident Commander.
- Keep a clear record of who made each decision.

DO NOT DO THIS

- ✗ One person handling everything.
- ✗ Unclear ownership.
- ✗ Too many people in the same channel.
- ✗ Making major changes without coordination.
- ✗ Focusing on root cause before restoring service.



SENIOR ENGINEER INSIGHT



Optimize first for limiting harm and restoring service. Deep analysis comes after the system is stable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



1 CONFIRM SIGNAL - Validate that this is a real incident. - Check multiple sources (metrics, logs, user reports). - Rule out false positives or expected events (e.g. maintenance).	2 ESTABLISH CUSTOMER IMPACT - Are customers affected? - How many users? - Which services? - Is it a partial or total outage? - Check user reports, SLOs and error rates.	3 DEFINE BLAST RADIUS - Which regions or AZs? - Which environments (prod, staging)? - Which services and dependencies? - Is the issue isolated or spreading?	4 CHECK RECENT CHANGES - Recent deployments - Configuration changes - Infrastructure changes - Feature flags - Database changes - DNS, certificates, IAM - Check change timeline (Last 24 hours).	5 ASSIGN INCIDENT OWNERSHIP - Declare an incident. - Assign an Incident Commander. - Assign key roles (Tech Lead, Comms, Scribe). - Ensure the right experts are involved.
6 OPEN COMMUNICATION CHANNELS - Create incident bridge (Slack, Teams, Zoom). - Set a single source of truth (incident channel). - Notify key stakeholders. - Share initial facts and next update time.	7 PRESERVE EVIDENCE - Capture logs, metrics and events. - Take snapshots if needed. - Record the timeline of events. - Preserve any relevant configuration or system state.	8 STOP RISKY CHANGES - Pause ongoing deployments. - Block non-essential changes. - Communicate a change freeze (if required). - Prevent further impact or configuration drift.	9 CREATE INITIAL HYPOTHESES - List possible causes based on evidence. - Prioritize the most likely causes. - Assign investigation tasks to team members. - Validate or disprove quickly using data.	10 DETERMINE IF MITIGATION IS REQUIRED - Is customer impact increasing? - Can the issue be contained? - Do you need immediate action to reduce harm? - Choose a safe and reversible mitigation if required.

FIRST 15 MINUTES CHECKLIST

1	Confirm the signal and validate the incident	☐
2	Establish customer impact and severity	☐
3	Define blast radius (services, regions, users)	☐
4	Check recent changes and timeline	☐
5	Assign incident ownership and key roles	☐
6	Open communication channels	☐
7	Preserve evidence (logs, metrics, events)	☐
8	Stop risky changes and enforce change freeze	☐
9	Create initial hypotheses and start investigation	☐
10	Decide if immediate mitigation is required	☐



SENIOR ENGINEER INSIGHT

In the first 15 minutes, optimize for limiting harm and restoring service. You can investigate the root cause in parallel, but customer impact reduction and stability take priority.



COMMON MISTAKES

- Assuming it is a false alarm without checking.
- Spending too much time on severity debates.
- Not involving the right experts early.
- Making changes without understanding the impact.
- Lack of clear communication.
- Trying to do everything alone.



GOOD PRACTICES

- Use data and evidence, not opinions.
- Keep communication clear and frequent.
- Make safe, reversible changes.
- Document decisions from the start.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 IDENTIFY WHAT IS FAILING



- Which services are failing?
- Which APIs or endpoints?
- Are errors, latency or timeouts increasing?
- Is it an application, platform or infrastructure issue?
- Check metrics, logs, traces and user reports.

2 FIND THE SCOPE



- Which regions are affected?
- Which Availability Zones (AZs)?
- Which clusters or environments?
- Which tenants or customers?
- Is the impact global, regional or isolated?
- Check if internal and external users are both affected.

3 TOTAL vs PARTIAL FAILURE

TOTAL FAILURE

- Service is completely unavailable
- All users affected
- No successful requests
- Usually SEV-1

PARTIAL FAILURE

- Some users or features affected
- Specific endpoints or data paths
- Degraded performance
- Could be SEV-2 or SEV-3

4 DEPENDENCY CORRELATION



- Check upstream and downstream services.
- Look for correlated failures.
- Identify shared dependencies (databases, queues, caches, third-party services).
- Validate whether the dependency is healthy.

5 HEALTHY vs UNHEALTHY PATHS



- Compare working and failing requests.
- Identify what is still working.
- Check different regions, clusters or tenants.
- Use a known-good path as a baseline.
- Helps isolate the failing component.

6 FAILURE-DOMAIN ISOLATION



- Is the failure in a single AZ, region, cluster or node?
- Is it limited to one service or spreading?
- Check network, DNS, load balancers and dependencies.
- Identify the blast radius to prevent further impact.

7 USE COMPARISON AS A DEBUGGING TECHNIQUE



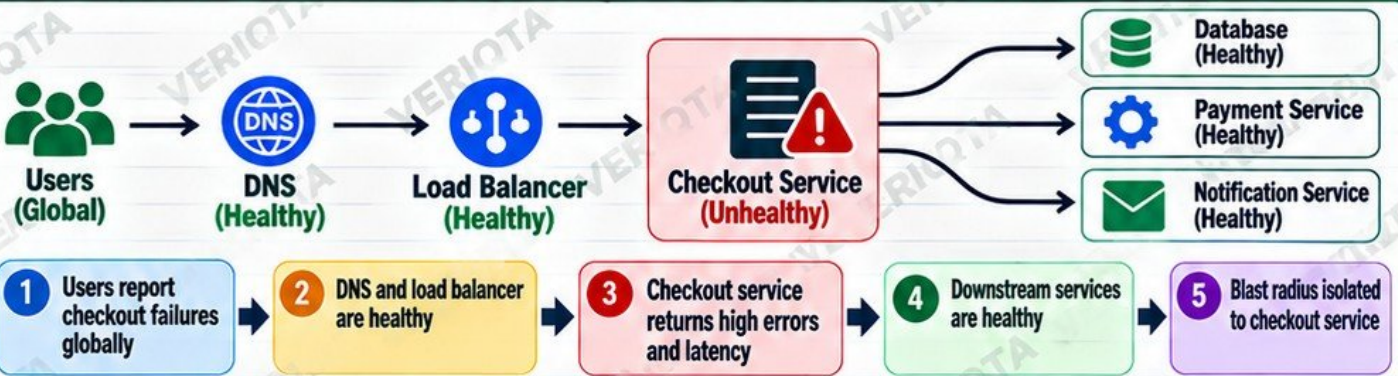
- Compare healthy vs unhealthy environments.
- Compare a working region with a failing region.
- Compare successful vs failed requests.
- Look for differences in configuration, dependencies or traffic.
- Helps quickly narrow the failure domain.



SENIOR ENGINEER INSIGHT

Start broad, then narrow. Use data to compare what is working and what is failing. Isolate the smallest possible failure domain before taking corrective action.

EXAMPLE: NARROWING GLOBAL SYMPTOMS TO THE AFFECTED COMPONENT



THE INVESTIGATION LOOP



1 FACTS vs ASSUMPTIONS

FACTS (Use evidence)	ASSUMPTIONS (Validate before acting)
- Based on observable data (metrics, logs, traces, events) - Verifiable and reproducible - Time-stamped - Supports decision-making	- Based on opinions or past experiences - Not yet proven - Can lead to wrong actions - Must be tested with evidence

2 HYPOTHESIS RANKING

Priority	Hypothesis	Why	Next Step
1	Recent deployment introduced a bug	Matches timeline and error pattern	Check deploy logs and diff
2	Database connection exhaustion	Errors and latency increased	Check DB metrics and connections
3	Dependency service failing	Increased timeouts in logs	Check dependency health
4	Network issue in one AZ	Partial user impact	Check network and AZ health

DISCONFIRMING EVIDENCE

- Actively look for evidence that proves your hypothesis wrong.
- Avoid only collecting data that confirms your belief.
- A strong investigation considers both supporting and disconfirming evidence.
- This reduces bias and leads to the real root cause.

PARALLEL INVESTIGATION

- Investigate multiple likely causes at the same time.
- Split tasks among team members.
- Compare findings regularly.
- Helps reduce time to root cause.
- Avoid everyone looking at the same thing.

AVOIDING CONFIRMATION BIAS

- Do not ignore data that does not fit your hypothesis.
- Be open to multiple possible causes.
- Use a structured approach and checklists.
- Involve other team members to get different perspectives.
- Let the evidence guide your conclusions.

MAINTAINING AN EVIDENCE TIMELINE

- Record key events in chronological order.
- Include alerts, user reports, changes, and findings.
- Use timestamps (UTC if possible).
- Helps identify patterns and correlations.
- Useful for communication and post-incident review.

7 EXAMPLE: BUILDING AN EVIDENCE TIMELINE

Time (UTC)	Event	Source	Findings
10:00	Alert: high error rate on checkout API	Monitoring	Error rate 12% (normal < 1%)
10:02	User reports checkout failures	Support	Multiple regions affected
10:05	Deployment completed	CI/CD	Version v2.3.1
10:08	Database latency increasing	Monitoring	Latency from 50ms to 5s
10:10	Dependency service timeouts	Logs	Timeouts to payment service
10:15	Hypothesis 1: recent deployment	Investigation	Checking code diff and logs

EXAMPLE: INVESTIGATION IN PRACTICE

Symptom: Checkout requests failing with 500 errors
Scope: Multiple regions (us-east-1, eu-west-1)
Timeline: Started at 10:00, after deployment at 09:55
Changes: New version v2.3.1 deployed
Signals: Increased errors, database latency, timeouts to payment
Hypotheses: 1) Deployment bug 2) DB connection exhaustion 3) Payment service failure
Tests: Check deployment logs, DB metrics, dependency health
Narrow: Deployment is fine, DB is healthy, payment service failing
Root Cause: Payment service outage causing cascading failures



SENIOR ENGINEER INSIGHT

Let the evidence tell the story. Stay open-minded, test your hypotheses, and be willing to change direction. The faster you find the truth, the faster you restore service.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 THE FOUR PILLARS OF OBSERVABILITY

METRICS	LOGS	TRACES	EVENTS
- Trends - Rates - Counts - Saturation - SLOs	- Detailed events - Errors - Application logs - System logs - Audit logs	- Request flow - Service hops - Latency - Dependencies - Find bottlenecks	- Alerts - Change events - Deployments - Infrastructure events - Security events

2 KEY SIGNALS TO CHECK

	Latency	Response time, p50 / p95 / p99
	Traffic	Request volume, RPS, trends
	Errors	Error rate, HTTP 4xx/5xx, exceptions
	Saturation	CPU, memory, disk, network, pod limits
	SLO burn rate	How quickly you are breaching SLOs
	Queue depth	Pending messages, backlog size
	Connection count	Active connections, pools, limits
	Retries and timeouts	Retry rate, timeout rate, failures
	Resource exhaustion	CPU, memory, disk, file descriptors

3 METRICS EXAMPLES

- Request rate (RPS)
- Latency (p50, p95, p99)
- Error rate (4xx, 5xx)
- CPU and memory usage
- Disk usage and IOPS
- Network in and out
- Saturation and throttling
- SLO / SLA metrics
- Custom business metrics (signups, checkouts, payments)

4 LOGS EXAMPLES

- Application logs
- Web server logs
- Error logs and stack traces
- System logs (journald)
- Kubernetes logs
- Database logs
- Security and audit logs
- Configuration change logs
- Look for errors, warnings and unusual patterns

5 TRACES EXAMPLES

- Request flow across services
- Service dependency map
- End-to-end latency
- Slowest service or call
- Failures and timeouts
- Retries and multiple attempts
- Which service is causing delay
- Compare successful vs failed requests

6 EVENTS EXAMPLES

- Deployments and rollbacks
- Infrastructure changes
- Scaling events
- Configuration changes
- Feature flag changes
- Database maintenance
- Security events
- Kubernetes events
- External dependency incidents

7 CORRELATION vs CAUSATION

CORRELATION

Two things happen at the same time.

- Error rate and latency increase together.
- CPU and response time rise together.
- Shows a possible relationship but not the actual cause.

CAUSATION

One thing directly causes the other.

- A database slowdown causes API timeouts.
- A memory leak causes pod restarts.
- Requires investigation and evidence to confirm.

8 WHY PRE-BUILT DASHBOARDS MATTER



- Designed before an incident to show the right signals.
- Faster identification of the affected service.
- Clear view of dependencies and blast radius.
- Shows SLOs, error rates, latency and saturation in real time.
- Reduces time spent searching for data.
- Helps compare healthy vs unhealthy paths.
- Provides a single source of truth for the team.
- Improvised investigation during an incident is slow and often misses important signals.

9 EXAMPLE: MULTI-SERVICE OBSERVABILITY DURING AN INCIDENT



10 INVESTIGATION CHECKLIST

- ☐ Check dashboards for latency, errors and traffic.
- ☐ Compare healthy vs unhealthy regions or services.
- ☐ Look at logs and traces for the time of the incident.
- ☐ Check infrastructure metrics (CPU, memory, disk, network).
- ☐ Review recent events and changes.
- ☐ Validate dependency health.
- ☐ Correlate signals across metrics, logs, traces and events.
- ☐ Identify the most likely failing component.

11 COMMON OBSERVABILITY QUESTIONS

- When did the issue start?
- Which services are affected?
- Is it global or limited to a region/AZ?
- Is the error rate increasing?
- Where is the latency coming from?
- Are dependencies failing?
- Is there resource saturation?
- Are we breaching SLOs?
- Do logs or traces show a clear error?
- What changed before the incident?

SENIOR ENGINEER INSIGHT

Good observability turns a chaotic incident into a structured investigation. Dashboards, logs, traces and events designed before an incident provide the right context, reduce mean time to recovery and help you focus on the real cause instead of guessing.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 TYPES OF CHANGES TO CHECK

	Deployments	Application releases and rollbacks
	Infrastructure changes	Servers, clusters, autoscaling, networking
	Feature flags	Feature enable/disable, gradual rollout
	Configuration	App config, environment variables
	Secrets	API keys, passwords, tokens, rotation
	Certificates	TLS certificates, expiration, rotation
	IAM	Permissions, roles, policy changes
	DNS	DNS records, routing, TTL changes
	Network policies	Firewalls, security groups, policies
	Database migrations	Schema changes, data migrations
	Dependency releases	Third-party services and libraries

2 WHERE TO FIND RECENT CHANGES

Change Type	Examples (Tools / Sources)
Deployments	CI/CD pipeline, deployment logs, Git history
Infrastructure	Terraform, CloudFormation, console audit logs
Feature flags	Feature-flag service, configuration dashboard
Configuration	Config management, environment variables
Secrets	Secrets manager, rotation logs, access logs
Certificates	Certificate manager, expiration alerts, cert logs
IAM	Cloud audit logs, IAM change history
DNS	DNS provider logs, change history, TTL updates
Network policies	Security groups, firewall logs, Kubernetes policies
Database migrations	Migration tools, database change logs
Dependency releases	Release notes, package manager logs

3 WHY "NOTHING CHANGED" IS RARELY ENOUGH

- Changes can happen outside your team.
- Infrastructure, dependencies or configuration may have changed.
- Rollouts, feature flags or scheduled jobs may have triggered.
- Third-party services you depend on may have released updates.
- Small changes can have a large impact.
- Always verify using evidence from multiple sources.

4 CHANGE CORRELATION TIPS

- Create a timeline of all changes.
- Compare change time with incident start time.
- Look for direct and indirect dependencies.
- Check for changes in all environments (prod, staging).
- Include third-party service changes.
- Look for rollouts and gradual feature releases.
- Validate whether the change aligns with the symptoms.

5 COMMON QUESTIONS

- ? What changed in the last 24 hours?
- ? Are there any deployments or rollbacks?
- ? Were there infrastructure or scaling changes?
- ? Were feature flags enabled or disabled?
- ? Did any configuration or secrets change?
- ? Did certificates expire or get rotated?
- ? Were IAM permissions modified?
- ? Were DNS or network policies updated?
- ? Were there database migrations?
- ? Did any third-party dependencies release updates?

6 EXAMPLE: INCIDENT TIMELINE WITH SYSTEM CHANGES



7 CHECKLIST: VERIFY RECENT CHANGES

- ☐ Review deployment history (application and infrastructure).
- ☐ Check configuration and environment variable changes.
- ☐ Look for feature-flag updates.
- ☐ Verify secrets and certificate rotations.
- ☐ Check IAM, DNS and network policy changes.
- ☐ Review database migrations and schema changes.
- ☐ Identify third-party dependency releases.
- ☐ Correlate change times with incident start time.
- ☐ Document your findings in the incident timeline.

8 GOOD PRACTICES

- ✓ Maintain a centralized change log.
- ✓ Use consistent timestamps (UTC).
- ✓ Include changes from all teams and tools.
- ✓ Track third-party service updates.
- ✓ Automate change correlation where possible.
- ✓ Keep change information easily searchable.
- ✓ Use change data to prioritize hypotheses.

SENIOR ENGINEER INSIGHT

Always assume something changed. "Nothing changed" is not evidence. Use data to confirm or rule out changes and correlate them with the incident timeline to find the real cause faster.

1 UPSTREAM vs DOWNSTREAM FAILURES

Upstream Dependency
(provides a service)

Failure propagates downstream

- Database
- Third-party API
- Authentication service
- Payment provider
- External DNS/TLS

Your Service
(consumes dependencies)

Failure propagates downstream

- Calls external services
- Handles errors and retries
- Can become a source of cascading failure

Downstream Services
(affected by your service)

- Internal microservices
- User-facing applications
- Background workers
- Batch jobs
- Queues and streams

2 COMMON FAILURE AREAS

	Timeouts	Requests take too long and eventually fail.
	Retries	Automatic retries can increase load.
	Backoff and jitter	Reduce retry storms and synchronized traffic.
	Circuit breakers	Stop calling a failing dependency after a threshold.
	Connection pools	Exhausted connections cause failures.
	DNS	Name resolution failures or slow responses.
	TLS	Certificate, handshake or encryption issues.
	Rate limiting	Requests are throttled by the dependency.

3 KEY INVESTIGATION QUESTIONS

- ? Which dependency is failing?
- ? Is the failure upstream or downstream?
- ? Are requests timing out or being rejected?
- ? Is retry logic increasing the load?
- ? Are connection pools exhausted?
- ? Is DNS resolving correctly?
- ? Are there any TLS or certificate issues?
- ? Are we being rate limited?
- ? Is this a partial or total failure?
- ? Are there cascading failures?
- ? How many services are affected?
- ? Did a recent change affect this dependency?

4 CASCADING FAILURES

- One service becomes slow or unavailable.
- Dependent services wait, retry and consume more resources.
- Increased load causes more timeouts and failures.
- Retries amplify the traffic (retry amplification).
- Resource exhaustion spreads across the platform.
- Multiple services and regions can be affected.

5 RETRY AMPLIFICATION

- When a dependency is slow, clients retry requests.
- Each retry adds more load to the failing service.
- Multiple services retry at the same time.
- This can quickly multiply traffic and make the incident worse.
- Use exponential backoff and jitter.
- Set reasonable retry limits.

6 CIRCUIT BREAKERS

- Prevent continuous requests to a failing dependency.
- Open the circuit after a failure threshold.
- Fail fast and return an error or fallback.
- Allow the dependency time to recover.
- Use monitoring to close the circuit when healthy.

7 EXAMPLE: ONE SLOW DEPENDENCY CREATES PLATFORM-WIDE SATURATION



8 INVESTIGATION CHECKLIST

- ☐ Identify all direct and indirect dependencies.
- ☐ Check metrics for latency, errors and request rate.
- ☐ Validate DNS resolution and network connectivity.
- ☐ Check TLS certificates and handshake errors.
- ☐ Review retry and timeout configuration.
- ☐ Inspect connection pool metrics.
- ☐ Look for rate limiting responses (e.g. 429).
- ☐ Check for cascading failures in downstream services.
- ☐ Compare healthy vs unhealthy regions or nodes.
- ☐ Correlate with recent changes.

9 COMMON METRICS TO CHECK

Metric	What It Tells You
Latency (p50/p95/p99)	Dependency is slow
Error rate (4xx/5xx)	Requests are failing
Request rate (RPS)	Volume and retry amplification
Connection count	Pool exhaustion
Timeout rate	Requests taking too long
Retry count	High retry traffic
Rate limit errors (429)	Being throttled
DNS lookup time	Name resolution issues
TLS handshake time	Certificate or network issues

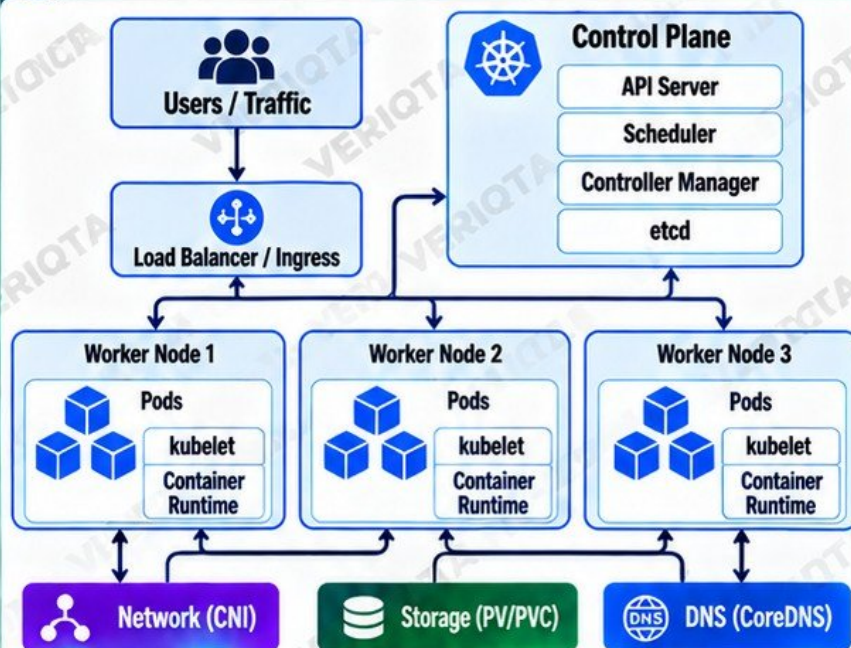
10 SENIOR ENGINEER INSIGHT

- 💡 A single slow or failing dependency can impact the entire platform. Always investigate dependencies early, control retries, use circuit breakers, and have clear observability across service boundaries.

1 COMMON INCIDENT AREAS

	Node conditions	NotReady, pressure, taints
	Pod health	Pending, CrashLoopBackOff
	OOMKilled	Out of memory termination
	CPU throttling	Limited CPU performance
	Scheduling failures	No nodes, taints, resources
	Evictions	Node pressure or resource limits
	DNS	Service discovery failures
	CNI / network failures	Pod to pod or external network
	Storage	PVC, PV, storage class issues
	Control-plane symptoms	API server, etcd, controller issues
	Rollout state	Failed or stuck deployments
	Resource saturation	CPU, memory, disk, network
	Kubernetes events	Warnings and errors

2 KUBERNETES ARCHITECTURE (INVESTIGATION VIEW)



3 KEY DIAGNOSTIC COMMANDS

Purpose	Command
List nodes	<code>kubectl get nodes -o wide</code>
Check node details	<code>kubectl describe node <node-name></code>
List pods	<code>kubectl get pods -A -o wide</code>
Check pod details	<code>kubectl describe pod <pod-name></code>
View pod logs	<code>kubectl logs <pod-name> [-c container]</code>
Check events	<code>kubectl get events -A --sort-by=.lastTimestamp</code>
Check resource usage	<code>kubectl top nodes</code> <code>kubectl top pods -A</code>
Check deployments	<code>kubectl get deploy -A</code> <code>kubectl rollout status deploy/<name></code>
Check services	<code>kubectl get svc -A</code>
Check endpoints	<code>kubectl get endpoints -A</code>
Check DNS	<code>kubectl get pods -n kube-system grep coredns</code>
Check storage	<code>kubectl get pv,pvc -A</code>
Check cluster info	<code>kubectl cluster-info</code>
Check control plane	<code>kubectl get componentstatuses</code>

4 COMMON PROBLEMS AND WHAT TO CHECK

Problem	Check
Node NotReady	<code>kubectl get nodes</code> , describe node, node events
Pod Pending	<code>kubectl describe pod</code> (look for scheduling events)
CrashLoopBackOff	<code>kubectl logs</code> , describe pod, previous logs
OOMKilled	<code>kubectl describe pod</code> (reason), check memory limits
CPU throttling	<code>kubectl top pods</code> , check CPU limits and metrics
Scheduling failures	Check resources, taints, node selectors, events
Pod evictions	Check node pressure, events, resource usage
DNS resolution issues	Check CoreDNS pods and logs, test DNS from pod
CNI / network issues	Check CNI pods, network policies, connectivity
Storage issues	Check PVC, PV, storage class, describe events
Control-plane issues	Check API server, etcd, controller manager logs
Rollout stuck	<code>kubectl rollout status</code> , describe deployment
High resource usage	<code>kubectl top</code> , check metrics and limits

5 INVESTIGATION CHECKLIST

- ☐ Check overall cluster and node health.
- ☐ Identify affected pods, services and namespaces.
- ☐ Review recent deployments and changes.
- ☐ Check Kubernetes events for warnings and errors.
- ☐ Validate resource usage (CPU, memory, disk, network).
- ☐ Check DNS and network connectivity.
- ☐ Inspect storage and persistent volumes.
- ☐ Review control-plane component status.
- ☐ Correlate findings with application logs and metrics.
- ☐ Identify the root cause before making changes.

6 PRODUCTION RULE



- Collect evidence before making changes.
- Understand the impact of any action.
- Avoid random restarts or rollbacks.
- Make small, safe, reversible changes.
- Document what you observe and do.

7 SENIOR ENGINEER INSIGHT

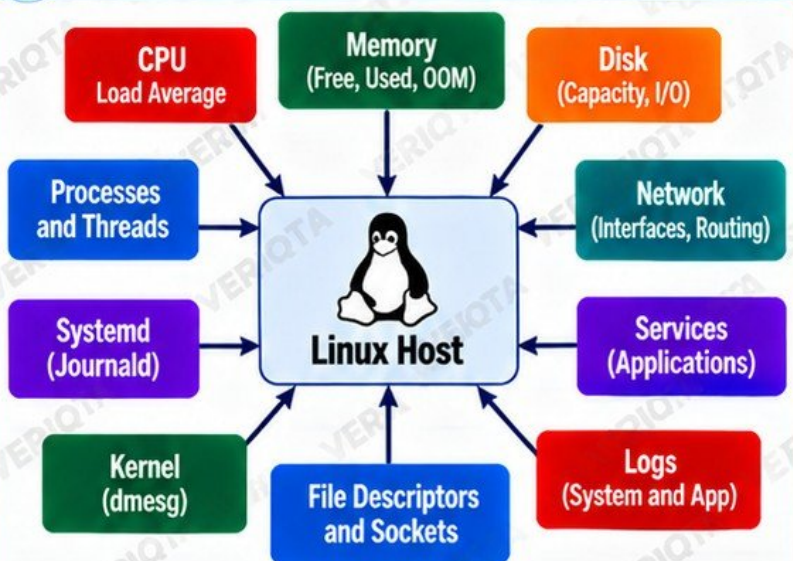


- ✓ Most Kubernetes incidents are caused by recent changes, resource saturation or dependencies.
- ✓ Always start with facts from the cluster.
- ✓ Use events, logs and metrics together.
- ✓ Look at the bigger picture: application, infrastructure and dependencies.
- ✓ A clear, calm investigation prevents bigger outages.

1 COMMON INCIDENT AREAS

	CPU saturation	High CPU usage, load average
	Memory pressure	Low memory, OOM behavior
	Disk capacity	Full disk, inode exhaustion
	I/O wait	Slow disk I/O, high iowait
	File descriptors	Too many open files
	Processes and threads	Unusual processes, thread leaks
	Sockets	High connections, socket states
	Systemd	Service failures and restarts
	Journald	System and application logs
	Kernel evidence	Kernel errors and warnings
	Network state	Interface, routing, connectivity

2 LINUX HOST OVERVIEW (INVESTIGATION VIEW)



3 CPU INVESTIGATION

Purpose	Command
Real-time CPU usage	top
Per-core CPU usage	mpstat -P ALL 1
Load average	uptime
Process CPU usage	ps aux --sort=-%cpu head
Detailed CPU stats	vmstat 1
Check for high system time	sar -u 1 5

4 MEMORY INVESTIGATION

Purpose	Command
Memory usage	free -h
Detailed memory stats	vmstat 1
Process memory usage	ps aux --sort=-%mem head
Check for OOM kills	dmesg grep -i oom
Memory usage by process	smem -r head
Swap usage	swapon --show

5 DISK AND I/O INVESTIGATION

Purpose	Command
Disk space usage	df -h
Inode usage	df -i
Disk I/O stats	iostat -xz 1
Process I/O usage	iotop
Block device info	lsblk
Disk usage by directory	du -sh /* 2>/dev/null sort -h

6 PROCESSES, THREADS AND FILE DESCRIPTORS

Purpose	Command
List processes	ps aux
Threads for a process	ps -eLf grep <pid>
Top memory/CPU processes	top or htop
Count open file descriptors	lsdf wc -l
Top file descriptor usage	lsdf -nP sort uniq -c sort -nr head
Check process limits	ulimit -a

7 SOCKETS AND NETWORK INVESTIGATION

Purpose	Command
Active connections	ss -tulnp
Connection states	ss -s
Listening ports	ss -ltnp
Network interfaces	ip a
Routing table	ip route
DNS resolution	dig <domain> nslookup <domain>
Check connectivity	ping <host> traceroute <host>

8 SYSTEMD AND LOG INVESTIGATION

Purpose	Command
Service status	systemctl status <service>
List failed services	systemctl --failed
Service logs	journalctl -u <service>
Recent system logs	journalctl -xe
Follow logs in real time	journalctl -f
Filter logs by time	journalctl --since "1 hour ago"
Kernel messages	dmesg

9 KERNEL AND OOM INVESTIGATION

Purpose	Command
View kernel messages	dmesg
Filter kernel errors	dmesg grep -i error
Check OOM events	dmesg grep -i oom
View hardware info	lscpu, lsblk, lspci
Check system limits	cat /proc/sys/fs/file-max
Check memory info	cat /proc/meminfo

COMMON INCIDENT SYMPTOMS

- High CPU and load average
- Out of memory or OOMKilled processes
- Full disk or high I/O wait
- Too many open files
- Service failures or restarts
- Network connectivity issues
- Unusual kernel errors
- Application errors in logs

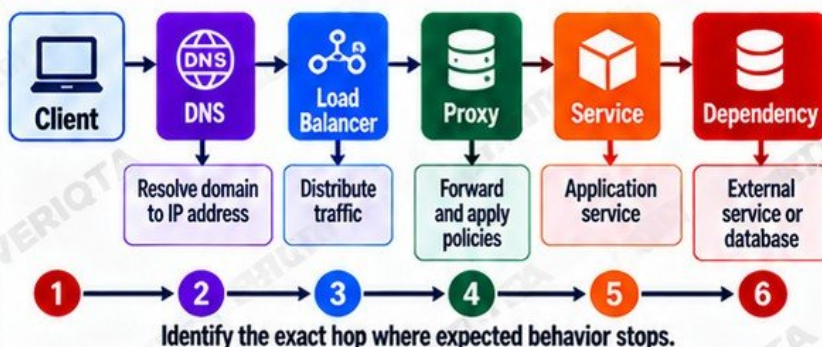
PRODUCTION RULE

- OBSERVE BEFORE CHANGING STATE**
- ✓ Collect and analyze evidence first.
 - ✓ Avoid unnecessary restarts or changes.
 - ✓ Understand the impact of any action.
 - ✓ Document your findings.
 - ✓ Make small, safe and reversible changes.
 - ✓ Always verify after making changes.

1 COMMON INCIDENT AREAS

	Request path reconstruction	Trace the full path from client to dependency
	DNS resolution	Name resolution failures or slow responses
	Routing	Incorrect routes or blackholing
	NAT	SNAT/DNAT issues, port exhaustion
	Firewalls	Blocked traffic or security rules
	Load balancers	Health check failures or misrouting
	Proxies	Forward proxy, reverse proxy issues
	TCP states	Connection states and resets
	Packet loss	Dropped packets or network congestion
	MTU	Fragmentation issues and large packets
	TLS handshake	Handshake failures or protocol issues
	Certificate expiration	Expired or invalid certificates
	Connection establishment	Timeouts or connection failures

2 REQUEST PATH RECONSTRUCTION (HIGH-LEVEL FLOW)



3 COMMON FAILURE POINTS BY HOP

Hop	What Can Go Wrong	Symptoms
Client → DNS	DNS not resolving or slow	Could not resolve host, high DNS latency
DNS → LB	Incorrect IP, routing issues	Connect timeout, wrong endpoint
LB → Proxy	Health check failures, blocked ports	503 errors, timeouts
Proxy → Service	Proxy misconfiguration, TLS issues	Connection reset, 502/504 errors
Service → Dependency	Network, DNS, firewall, TLS, overload	Timeouts, connection refused, slow responses

4 DNS INVESTIGATION

Purpose	Command
Resolve domain	<code>dig <domain></code>
Detailed query	<code>dig <domain> +trace</code>
Check resolution	<code>nslookup <domain></code>
Check all records	<code>dig <domain> any</code>
Test from specific DNS	<code>dig <domain> @8.8.8.8</code>
Reverse lookup	<code>dig -x <ip></code>
Continuous check	<code>watch -n 2 dig <domain></code>

5 NETWORK AND ROUTING INVESTIGATION

Purpose	Command
Test connectivity	<code>ping <host></code>
Trace network path	<code>tracert <host></code>
Show route table	<code>ip route</code>
Check interface status	<code>ip a</code>
Check NAT connections	<code>sudo conntrack -L</code>
Check firewall rules (Linux)	<code>sudo iptables -L</code> <code>sudo nft list ruleset</code>
Test MTU	<code>ping -M do -s 1472 <host></code>
Check for packet loss	<code>mtr <host></code>

6 LOAD BALANCER AND PROXY

Purpose	Command
Check LB health	<code>curl -I <lb-dns></code>
Test specific backend	<code>curl -I <backend-ip></code>
Inspect headers	<code>curl -v <url></code>
Check proxy config	<code>cat /etc/nginx/nginx.conf</code>
View proxy logs	<code>tail -f /var/log/nginx/access.log</code>
Check connection count	<code>ss -s</code>
Check open connections	<code>ss -tnp grep :80</code>

7 TLS INVESTIGATION

Purpose	Command
Check certificate	<code>openssl s_client -connect <host>:443 -servername <host></code>
View certificate details	<code>openssl x509 -in cert.pem -text</code>
Check expiry date	<code>openssl x509 -in cert.pem -enddate -noout</code>
Test TLS protocol	<code>openssl s_client -connect <host>:443 -tls1_2</code>
Check supported ciphers	<code>openssl ciphers -v</code>
Check with curl	<code>curl -v https://<host></code>

8 TCP STATES AND CONNECTIONS

Purpose	Command
View TCP connections	<code>ss -tulnp</code>
View connection states	<code>ss -tan state established</code>
Check for TIME_WAIT	<code>ss -tan state time-wait</code>
Count connections	<code>ss -s</code>
Find connections to host	<code>ss -tnp grep <host></code>
Check listening ports	<code>ss -lntp</code>
Check socket usage	<code>lsof -i -nP</code>

9 COMMON ERROR SYMPTOMS

- Could not resolve host
- Name or service not known
- Connection timed out
- Connection refused
- TLS handshake failed
- certificate has expired
- no route to host
- network is unreachable
- 502 / 503 / 504 errors
- packet loss or high latency
- Intermittent connectivity

10 DEBUGGING CHECKLIST

- ☐ Reconstruct the full request path from client to dependency.
- ☐ Test each hop independently.
- ☐ Verify DNS resolution and TTL.
- ☐ Check routing, NAT and firewall rules.
- ☐ Validate load balancer and proxy configuration.
- ☐ Confirm TCP connectivity and MTU.
- ☐ Inspect TLS certificates and handshake.
- ☐ Check for packet loss, latency and connection resets.
- ☐ Identify the exact hop where expected behavior stops.
- ☐ Document findings with evidence (commands and outputs).

11 SENIOR ENGINEER INSIGHT

- Network, DNS and TLS incidents are often multi-layered. Always test each hop independently and collect concrete evidence before making changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

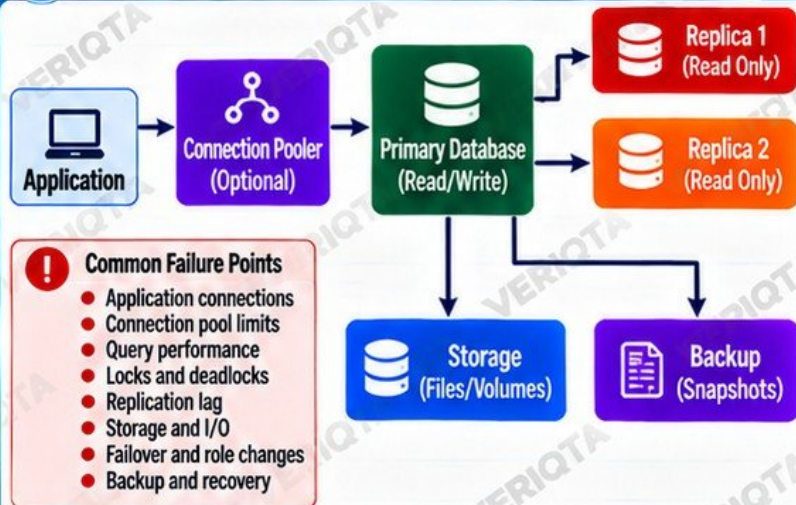


@veriqta

1 COMMON INCIDENT AREAS

	Connection exhaustion	Too many active connections
	Slow queries	High latency, long execution time
	Lock contention	Sessions waiting for locks
	Deadlocks	Circular lock dependencies
	Replication lag	Delay between primary and replica
	Storage pressure	High disk usage, slow I/O
	CPU and memory saturation	High resource usage
	Failover	Unexpected or manual failover
	Primary/replica behavior	Role changes, read/write issues
	Data integrity	Risk of data loss or corruption
	Recovery risk	Long recovery time and impact

2 TYPICAL DATABASE ARCHITECTURE



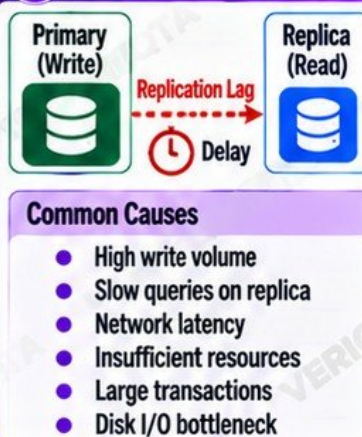
3 KEY SYMPTOMS AND WHAT THEY MEAN

Symptom	Possible Cause
Too many connections	Application issue, missing pooling, leaks
Slow queries	Inefficient queries, missing indexes, high load
Lock waits or deadlocks	Lock contention, long transactions
Replication lag	Slow queries, high write load, network issues
High disk usage	Large data, unpurged logs, lack of cleanup
High CPU usage	Expensive queries, full table scans
High memory usage	Large caches, memory leaks
Frequent failovers	Instability, infrastructure or configuration issues
Inconsistent data	Replication issues, failed writes, corruption risk

4 USEFUL DIAGNOSTIC COMMANDS

Purpose	Example Command (Linux / SQL)
Check active connections	<code>SELECT * FROM pg_stat_activity;</code>
Find long-running queries	<code>SELECT * FROM pg_stat_activity WHERE state = 'active' ORDER BY query_start;</code>
View locks	<code>SELECT * FROM pg_locks;</code>
Check replication status	<code>SELECT * FROM pg_stat_replication;</code>
Check database size	<code>SELECT pg_size_pretty(pg_database_size('dbname'));</code>
Check indexes	<code>SELECT * FROM pg_indexes;</code>
Monitor disk usage	<code>df -h</code>
Check I/O performance	<code>iostat -xz 1</code>
Check CPU and memory	<code>top / htop</code>
View logs	<code>tail -f /var/log/postgresql.log</code>

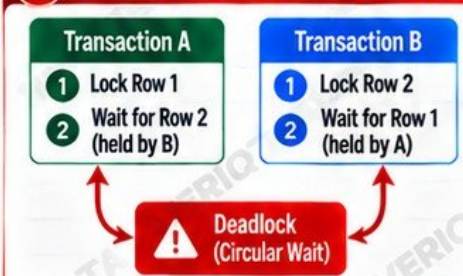
5 REPLICATION LAG ANALYSIS



6 CONNECTION EXHAUSTION



7 LOCK CONTENTION AND DEADLOCKS



8 DISK, CPU AND MEMORY PRESSURE

	Disk	- High disk usage - Slow I/O - Full tablespace
	CPU	- High CPU usage - Expensive queries - Too many connections
	Memory	- High memory usage - Large caches - Memory leaks - OOM (Out of Memory)

9 WHY RESTARTING A DATABASE CAN HIDE SYMPTOMS

	HIDES THE REAL ISSUE		INCREASES OPERATIONAL RISK
- Clears memory and connections - Stops current queries and locks - Temporary improvement in performance - Original cause still exists - Issue often returns and can be worse		- Possible data loss if not graceful - Longer recovery time - Replication disruption - Service downtime - Can make troubleshooting harder	

10 INVESTIGATION CHECKLIST

- ☐ Check active connections and limits.
- ☐ Identify slow queries.
- ☐ Look for locks and deadlocks.
- ☐ Check replication lag and replica health.
- ☐ Verify disk, CPU and memory usage.
- ☐ Review logs for errors and warnings.
- ☐ Validate backup status.
- ☐ Assess failover and recovery risk.
- ☐ Confirm data integrity.
- ☐ Avoid unnecessary restarts.

11 SENIOR ENGINEER INSIGHT

A database restart may temporarily improve performance by clearing connections and memory, but it can hide the real problem, increase operational risk and lead to data loss or longer recovery time. Always investigate and fix the root cause instead of relying on restarts.

1 MITIGATION vs REMEDIATION



MITIGATION

Reduce immediate harm.

- Stabilize the system.
- Restore core functionality.
- Buy time for a full fix.
- Often temporary.
- Focus on safety and user impact.



REMEDiation

Correct the underlying defect.

- Fix the root cause.
- Make a permanent change.
- Prevent recurrence.
- Often requires planning and testing.
- May be done after the incident is stabilized.

2 MITIGATION FLOW (HIGH-LEVEL)



3 COMMON MITIGATION STRATEGIES



Rollback

- Revert to a previous stable version.
- Used for bad deployments or configuration changes.



Feature Disabling

- Turn off non-critical features.
- Reduce load or avoid triggering the issue.



Traffic Shifting

- Route traffic to healthy regions or clusters.
- Use load balancers, DNS or service mesh.



Capacity Increases

- Scale up compute, memory or database capacity.
- Add more nodes or replicas.
- Useful for temporary spikes.



Rate Limiting

- Limit incoming requests.
- Protect downstream services.
- Prevent further saturation.



Dependency Isolation

- Disable or bypass a failing dependency.
- Use fallbacks or mocks.
- Keep core services running.



Failover

- Switch to a standby instance or region.
- Used for database, cache or critical services.



Load Shedding

- Intentionally drop non-critical requests.
- Protect essential functionality.
- Return a friendly error or queue the request.



Graceful Degradation

- Serve a reduced version of the service.
- Disable expensive operations.
- Maintain core user experience.



Temporary Configuration Changes

- Adjust timeouts, connection limits or retry settings.
- Change resource limits.
- Tweak load balancer or cache settings.



Risk Assessment Before Mitigation

- Understand potential impact.
- Consider data loss, downtime and user experience.
- Validate the change in a safe way if possible.



Important

- Mitigation should be fast, safe and reversible.
- Always communicate changes.
- Document what you do and why.

4 EXAMPLE: TRAFFIC SHIFTING DURING AN INCIDENT



5 MITIGATION vs REMEDIATION EXAMPLE

MITIGATION (During Incident)

- Roll back to previous version.
- Disable a non-critical feature.
- Increase capacity.
- Apply rate limiting.
- Shift traffic to a healthy region.

REMEDiation (After Stabilization)

- Fix the bug in the new version.
- Improve tests and validation.
- Update monitoring and alerts.
- Redesign to handle traffic spikes.
- Document the incident and prevent recurrence.

6 MITIGATION CHECKLIST

- ☐ Confirm the impact and scope.
- ☐ Identify safe mitigation options.
- ☐ Assess risks and side effects.
- ☐ Communicate the plan to stakeholders.
- ☐ Implement the change carefully.
- ☐ Monitor the results.
- ☐ Adjust if the issue is not fully resolved.
- ☐ Document what you did and the outcome.

7 COMMON PITFALLS

- Making changes without understanding the impact.
- Restarting services without analysis.
- Disabling critical functionality.
- Over-scaling and increasing cost unnecessarily.
- Ignoring data integrity risks.
- Applying multiple changes at the same time.
- Not communicating with the team.
- Forgetting to follow up with proper remediation.

8 SENIOR ENGINEER INSIGHT



- Mitigation is about controlling the damage, not fixing everything.
- Choose the smallest effective change.
- Prefer reversible changes.
- Always verify the impact after the change.
- Plan and execute full remediation once the system is stable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 WHEN ROLLBACK IS SAFEST

- Recent, isolated change.
- No database schema changes.
- Backward compatible deployment.
- Configuration or version change only.
- Clear previous stable version.
- Low risk of data inconsistency.
- Quick rollback process available.

2 WHEN ROLLBACK CANNOT WORK

- Database schema migrations that are not backward compatible.
- Data has already been modified in an irreversible way.
- Multiple dependent services have changed.
- Rolling back will cause further outages or data loss.
- External dependencies have also changed.
- Long time since the change (environment drift).

3 DATABASE MIGRATION COMPLICATIONS



- Schema changes are not easily reversible.
- Data transformations may be one-way.
- Rolling back application code may not work with new schema.
- Requires careful planning and migration strategies (expand and contract).
- May need a forward fix instead of rollback.
- Consider backups and point-in-time recovery.

4 FORWARD FIX vs ROLLBACK



ROLLBACK

- Revert to a previous version.
- Faster when safe to use.
- Works well for code and configuration changes.
- May not be possible for database or data changes.



FORWARD FIX

- Deploy a new version that fixes the issue.
- Required when rollback is not possible.
- May take longer to develop and test.
- Reduces risk of data loss in complex scenarios.

5 FAILOVER STRATEGIES



ACTIVE-ACTIVE

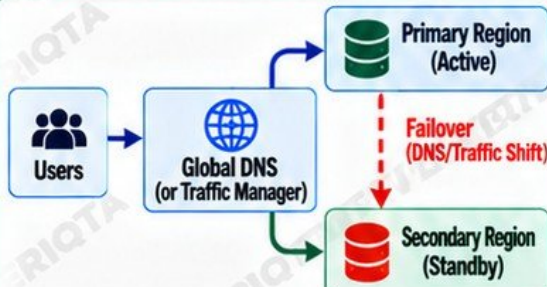
- Multiple regions are active at the same time.
- Traffic is distributed across regions.
- Higher availability and capacity.
- More complex and higher cost.
- Requires data consistency across regions.



ACTIVE-PASSIVE

- One primary region is active.
- Secondary region is on standby.
- Failover happens during an outage.
- Simpler and lower cost.
- Requires testing to ensure fast failover.
- Consider data replication and consistency.

6 REGIONAL FAILOVER



7 STATE CONSISTENCY



- Keep data in sync across regions or replicas.
- Understand replication model (synchronous or asynchronous).
- Be aware of replication lag.
- Validate data integrity after failover.
- Consider conflict resolution in active-active setups.

8 RPO AND RTO



RPO (Recovery Point Objective)

- Maximum acceptable data loss (measured in time).
- Depends on backup and replication strategy.



RTO (Recovery Time Objective)

- Maximum acceptable downtime (measured in time).
- Influences infrastructure, automation and failover process.

9 DECISION MATRIX

Action	Benefit	Risk	Reversibility	Verification
Rollback to previous version	Quick recovery	May not work if data changed	High	Check service health, error rates, functionality
Forward fix (new deployment)	Addresses root cause	Longer time, new risks	Medium	Verify tests, monitor metrics and logs
Feature disablement	Stops impact quickly	Reduced functionality	High	Confirm traffic reduction
Traffic shifting (to healthy region)	Maintains service availability	Potential data inconsistency	High	Check user experience, metrics, error rates
Capacity increase (scale up/out)	Handles high load	Higher cost, may not fix root cause	Medium	Confirm resource usage and performance
Failover to secondary region	Recovers from major outage	Data lag, service disruption	Medium	Validate service health and data consistency
Load shedding	Protects critical services	Reduced features for users	High	Verify critical paths remain healthy
Dependency isolation	Prevents cascading failures	Reduced functionality	High	Confirm downstream stability
Temporary configuration change	Quick mitigation	Unexpected side effects	High	Monitor metrics, logs and behavior

10 RECOVERY CHECKLIST

- ☐ Understand the scope and impact.
- ☐ Choose the safest recovery action.
- ☐ Validate data consistency.
- ☐ Communicate with stakeholders.
- ☐ Execute rollback, failover or forward fix.
- ☐ Monitor service health and user experience.
- ☐ Verify that the issue is resolved.
- ☐ Document what was done.

11 SENIOR ENGINEER INSIGHT



- Rollback is safest for simple code and configuration changes.
- Use forward fixes for complex changes like database migrations.
- Regularly test your failover process.
- Always consider data consistency, RPO and RTO before making a decision.
- Choose the option with the lowest risk to the business and users.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 AUDIENCE AND PURPOSE



Technical Updates

- Detailed and technical information.
- Focus on investigation, mitigation and next steps.
- For engineers and technical teams.



Executive Updates

- High-level, clear and non-technical.
- Focus on business impact, mitigation status and next steps.
- For leadership and stakeholders.



Customer-Facing Updates

- Clear, simple and accurate.
- Focus on impact, mitigation and expected next update.
- For customers, end users and the public.

2 KEY INFORMATION TO INCLUDE



Known Facts

Verified and confirmed information.



Hypotheses

Possible causes, clearly labeled as not yet confirmed.



Impact Statements

What is affected and the severity.



Mitigation Status

Actions taken and current status.



Next Update Expectations

When the next update will be provided.

3 IMPORTANT GUIDELINES



Avoid speculative root-cause statements.



Share only verified information as facts.



Be clear, concise and consistent across all channels.



Use simple language for customer updates.



Maintain a regular communication cadence.



Update even if there is no significant change.



Correct misinformation quickly and transparently.

4 COMMUNICATION CADENCE



Initial Update

As soon as impact is confirmed (within minutes).



Regular Updates

Provide updates at consistent intervals (e.g. every 30-60 minutes).



Status Change

Update when there is a material change (new impact, mitigation progress, or resolution).



Final Update

Confirm resolution and next steps (e.g. post-incident review).

5 EXAMPLE CONCISE INCIDENT UPDATE STRUCTURE



Title

Brief summary of the incident.



Time

When the update was published.



Current Status

Ongoing, mitigated or resolved.



Impact

What is affected and the severity.



What We Are Doing

Mitigation steps in progress.



Next Update

When the next update will be provided.



Additional Notes

Any important context or actions for users.

6 TECHNICAL UPDATE EXAMPLE

Title: API latency increase

Time: 2026-09-25 14:30 UTC

Current Status: Investigating

Impact: Elevated latency on the payment API (p95 ~ 2s).

What We Know: Increased error rate and higher database response time.

What We Are Doing: Checking database performance, reviewing recent deployment, scaling read replicas.

Next Update: In 30 minutes or sooner if there is a significant change.

Additional Notes: No data loss detected at this time.

7 EXECUTIVE UPDATE EXAMPLE

Title: Partial service disruption

Time: 2026-09-25 14:30 UTC

Current Status: Mitigating

Impact: Some customers are experiencing slower response times. Core business functions remain available.

What We Are Doing: Engineering is implementing mitigation steps and monitoring the impact.

Next Update: In 30 minutes.

Additional Notes: We will provide a full report after resolution.

8 CUSTOMER-FACING UPDATE EXAMPLE

Title: Service disruption update

Time: 2026-09-25 14:30 UTC

Current Status: Investigating

Impact: Some users may experience slower response times.

What We Are Doing: Our team is actively working on a fix.

Next Update: In 30 minutes.

Additional Notes: We will share another update as soon as there is new information.

9 COMMUNICATION CHECKLIST



Confirm the facts before sharing.



Identify the right audience.



Use clear and simple language.



Include impact and mitigation status.



Set expectation for the next update.



Be consistent across all channels.



Avoid speculation.



Keep to a regular cadence.



Update even if there is no change.



Document all communications.

10 SENIOR ENGINEER INSIGHT



- Good incident communication builds trust.
- It reduces confusion and speculation.
- It helps stakeholders make better decisions.
- Clear, consistent and honest updates are as important as the technical fix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 KEY PRINCIPLE



Recovery is not simply "the graph is green."

A service is only considered recovered when you have verified functionality, performance, data correctness and no hidden secondary failures.

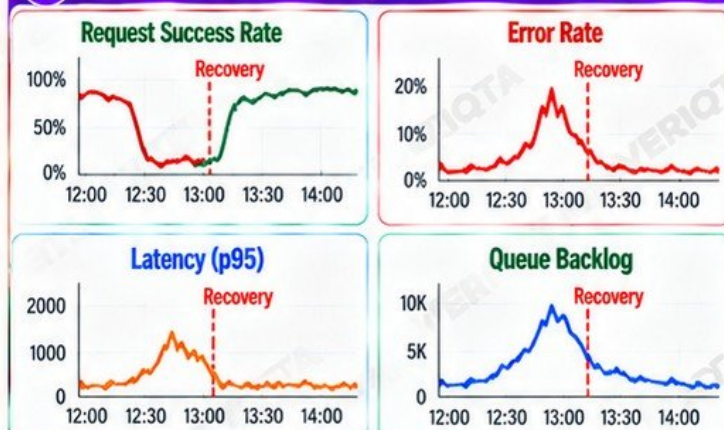
2 RECOVERY DECISION GATE



3 WHAT TO VERIFY AFTER RECOVERY

Area	What to Check
Customer requests	Requests succeed, no user-facing errors.
Error rate	Error rate back to normal levels.
Latency	Response time within acceptable range.
Saturation	CPU, memory, disk and network utilization to normal levels.
Queue backlog	Queues processed, no growing backlog.
Dependency health	All upstream and downstream dependencies healthy.
Data correctness	Verify no data loss or corruption.
Replication	Replication caught up and in sync.
Background jobs	Scheduled jobs and workers running successfully.
Autoscaling	Auto scaling working as expected.
Capacity headroom	Sufficient headroom for unexpected traffic.
SLO recovery	SLOs back within the defined targets.
Synthetic tests	Run end-to-end synthetic checks.
No hidden failures	Check for secondary failures in related services.

4 EXAMPLE METRICS BEFORE AND AFTER RECOVERY



5 VERIFICATION CHECKLIST

- | | |
|--|---|
| ☐ Customer requests are successful.
☐ Error rate is normal.
☐ Latency is within target.
☐ Resource utilization is normal.
☐ Queues are processed.
☐ All dependencies are healthy.
☐ Data is correct and complete. | ☐ Replication is in sync.
☐ Background jobs are running.
☐ Autoscaling is functioning.
☐ There is enough capacity headroom.
☐ SLOs have recovered.
☐ Synthetic tests pass.
☐ No hidden secondary failures. |
|--|---|

6 EXAMPLE RECOVERY TESTS

Test	Example Command or Check
Check service endpoints	<code>curl -I https://your-service.com</code>
Check error rate (Prometheus)	<code>rate(http_requests_total{status=~"5.."}[5m])</code>
Check latency (Prometheus)	<code>histogram_quantile(0.95, rate(http_request_duration_seconds[5m]))</code>
Check queue backlog	<code>aws sqs get-queue-attributes --queue-url <url> --attribute-names ApproximateNumberOfMessages</code>
Check replication (PostgreSQL)	<code>SELECT * FROM pg_stat_replication;</code>
Check background jobs	<code>kubectl get jobs; kubectl get cronjobs</code>
Check autoscaling	<code>kubectl get hpa</code>
Run synthetic test	<code>curl -s https://your-service.com/health grep "OK"</code>

7 COMMON POST-RECOVERY ISSUES

- Hidden failures in related services.
- Data inconsistency or incomplete replication.
- Queues still processing old messages.
- Slow recovery of caches or indexes.
- Autoscaling not returning to normal.
- Resource exhaustion after traffic returns.
- Recurring errors due to incomplete fix.
- SLOs not fully recovered.
- Customers still experiencing issues in certain regions.

8 SENIOR ENGINEER INSIGHT

- Do not rely only on metrics looking green.
- Verify from the user's perspective.
- Check data and background processes.
- Look for secondary and regional impacts.
- Allow enough observation time before closing.
- A true recovery restores functional, reliable and sustainable operation.

9 RECOVERY VALIDATION FLOW



1 PURPOSE AND MINDSET



- Understand what happened.
- Find real root causes.
- Identify gaps and improvements.
- Prevent recurrence.
- Blameless and fact-based analysis.
- Focus on systems, not individuals.

2 TIMELINE RECONSTRUCTION

- 10:00 Normal operation
- 10:15 Trigger event
- 10:18 First user impact
- 10:22 Detection (alert)
- 10:25 Response started
- 10:40 Mitigation applied
- 11:10 Service recovered

Include in timeline:

- Events (deployments, changes)
- Alerts and detections
- User impact
- Actions taken
- Communication updates
- Recovery time

3 INCIDENT ANALYSIS AREAS

- Trigger** What started the incident?
- Contributing Conditions** What factors made the impact worse?
- Detection Gaps** Why was it not detected earlier or faster?
- Response Gaps** What slowed down mitigation or recovery?
- Technical Root Causes** What directly caused the failure?
- Organizational Contributors** Process, design or cultural factors.

4 EXAMPLE INCIDENT TIMELINE

Time (UTC)	Event	Details
10:00	Normal operation	All services healthy
10:15	Trigger	New version deployed
10:18	User impact	Increased error rate
10:20	Alert	High 5xx errors detected
10:22	Investigation	Identified failing service
10:40	Mitigation	Rolled back to previous version
11:10	Recovery	Error rate back to normal
11:30	Validation	All checks passed

5 FIVE WHYS (EXAMPLE)

- Why did the service fail? → Database connections were exhausted.
 - Why were connections exhausted? → A traffic spike increased connection usage.
 - Why did the spike cause exhaustion? → Connection pool size was too small.
 - Why was the pool size too small? → It was not updated for expected load.
 - Why was it not updated? → Capacity planning and load testing were missing.
- Limitation:** Five Whys is useful but often incomplete. Complex incidents usually have multiple root causes and contributing factors.

6 WHY "HUMAN ERROR" IS INCOMPLETE



- Humans work within systems.
- Human error is usually a symptom of deeper issues.
- Often caused by unclear processes, poor tooling, lack of training, time pressure or unsafe design.



- Look for systemic and organizational factors, not just individual mistakes.
- Focus on how to make the system more resilient to human mistakes.

7 ROOT CAUSE CATEGORIES

Category	Examples
Technical Root Cause	Code bug, misconfiguration, resource limit, dependency failure.
Contributing Conditions	Traffic spike, unusual data, third-party issue, infrastructure issue.
Detection Gaps	Missing alerts, incorrect thresholds, no visibility.
Response Gaps	Delayed response, unclear runbooks, lack of ownership.
Organizational Factors	Process gaps, poor change management, insufficient testing, knowledge silos.

8 COUNTERFACTUAL REASONING

- “What would have happened if we had done X?”
- Helps validate root cause and contributing factors.
- Tests whether proposed fixes would have prevented or reduced the impact.
- Avoid hindsight bias.
- Use realistic and evidence-based scenarios.

9 EVIDENCE-BACKED CONCLUSIONS



- Use logs, metrics, traces, events and system data.
- Avoid assumptions without evidence.
- Clearly separate facts from hypotheses.
- State the primary root cause and supporting causes.
- Be specific and actionable.
- Link each conclusion to evidence.

10 POST-INCIDENT REVIEW CHECKLIST

- ☐ Reconstruct detailed timeline.
- ☐ Identify trigger and root causes.
- ☐ List contributing conditions.
- ☐ Find detection and response gaps.
- ☐ Include organizational factors.
- ☐ Validate conclusions with evidence.
- ☐ Define and track action items.
- ☐ Assign owners and due dates.
- ☐ Review similar risks in other services.
- ☐ Update alerts, runbooks and processes.
- ☐ Share lessons with relevant teams.
- ☐ Follow up to ensure actions are completed.

11 SENIOR ENGINEER INSIGHT



- The goal is to prevent the next incident, not to find someone to blame.
- A good post-incident review makes the system stronger, simpler and more resilient.
- Look for patterns across multiple incidents.
- Use clear, honest and factual analysis.
- Actionable improvements are more important than a perfect root cause analysis.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1 CORE PRINCIPLE



- Move beyond "add monitoring."
- Address the real root causes and system weaknesses.
- Implement meaningful, measurable and verifiable actions.
- Prevent recurrence and improve overall reliability.

2 ACTION CLASSIFICATION

	Prevention	Eliminate or reduce the chance of the issue happening again.
	Detection	Identify the issue earlier.
	Containment	Limit the blast radius.
	Mitigation	Reduce the impact.
	Recovery	Restore service faster.
	Documentation	Improve runbooks and knowledge.
	Automation	Automate repetitive and critical tasks.
	Testing	Add tests to catch issues.
	Architecture	Improve system design.
	Operational Process	Improve processes, training and ownership.

3 EXAMPLES FOR EACH CATEGORY

	Prevention	• Input validation • Safer deployment process
	Detection	• Better alerting • Synthetic checks
	Containment	• Rate limiting • Isolation of failing service
	Mitigation	• Traffic shifting • Feature disablement
	Recovery	• Faster failover • Automated recovery process
	Documentation	• Update runbooks • Document lessons learned
	Automation	• Auto remediation • Self healing mechanisms
	Testing	• Add integration tests • Chaos and failure testing
	Architecture	• Remove single points of failure • Improve scalability
	Operational Process	• Clear ownership • Better change management

4 ACTION TRACKING TEMPLATE

Action	Category	Owner	Priority	Due Date	Verification Criteria
Implement input validation	Prevention	Backend Team	High	2026-10-15	Validated in staging and production
Add database connection alerts	Detection	SRE Team	High	2026-10-10	Alerts triggered in test
Implement rate limiting	Containment	Platform Team	Medium	2026-10-20	Verified under load test
Automate failover	Recovery	Infrastructure Team	High	2026-10-25	Successful failover during test
Update incident runbook	Documentation	Tech Lead	Medium	2026-10-12	Reviewed and published

5 ACTION OWNERSHIP AND PRIORITY

	Owner	• Person or team responsible for completing the action. • Must be clearly assigned.
	Priority	• High Critical for risk reduction. • Medium Important but not blocking. • Low Nice to have or long term.
	Due Date	• Set a realistic and specific date. • Track progress regularly.
	Verification Criteria	• Define how you will confirm that the action is complete and effective. • Use measurable outcomes.

6 GOOD vs WEAK CORRECTIVE ACTIONS

	GOOD ACTIONS		WEAK ACTIONS
	Addresses the actual root cause.		Vague or generic.
	Specific and measurable.		Only "add monitoring."
	Has a clear owner and due date.		No owner or due date.
	Includes verification criteria.		Cannot be measured.
	Reduces the risk of recurrence.		Does not address the root cause.
			Unlikely to prevent recurrence.

7 COMMON CORRECTIVE ACTIONS

- Fix code or configuration issues.
- Improve validation and error handling.
- Add or improve alerts and runbooks.
- Implement automated rollback.
- Increase test coverage.
- Improve deployment process.
- Remove single points of failure.
- Add capacity or scaling improvements.
- Update documentation and diagrams.
- Provide training where needed.

8 CHECKLIST BEFORE CLOSING

- ☐ All corrective actions are identified.
- ☐ Owners are assigned.
- ☐ Priorities are set.
- ☐ Due dates are defined.
- ☐ Verification criteria are clear.
- ☐ Progress is tracked.
- ☐ Actions are completed and verified.
- ☐ Residual risks are documented.

9 SENIOR ENGINEER INSIGHT



A postmortem without completed corrective actions is documentation, not risk reduction.

10 KEY TAKEAWAYS

- Corrective actions prevent the next incident.
- Focus on system and process improvements.
- Be specific, measurable and accountable.
- Involve the right stakeholders.
- Track and verify completion.
- Use lessons learned to improve reliability across the organization.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



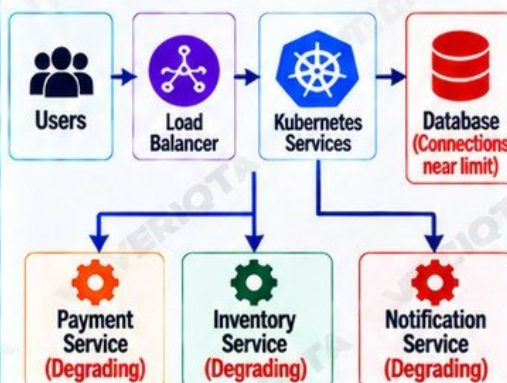
@veriqta

1 SCENARIO OVERVIEW



Checkout latency rises from 300 ms to **8 seconds**.
Error rates increase.
Kubernetes CPU looks normal.
Database connections approach maximum. Retry volume triples.
A deployment occurred 20 minutes earlier.
Multiple downstream services begin degrading.

2 ARCHITECTURE DIAGRAM



3 KEY OBSERVATIONS

-  Latency: 300 ms → **8 seconds**
-  Error rates: increasing
-  Kubernetes CPU: normal
-  Database connections: approaching maximum
-  Retry volume: triples
-  Deployment: 20 minutes earlier
-  Multiple downstream services: degrading

4 INCIDENT INVESTIGATION QUESTIONS

- 1 Determine initial severity and blast radius.
- 2 What is the first evidence to collect?
- 3 Which roles should be assigned?
- 4 What are the highest-value hypotheses?
- 5 How may retries amplify the incident?
- 6 What is the safest immediate mitigation?
- 7 What are the rollback risks?
- 8 What signals are required before declaring recovery?
- 9 How should you communicate with stakeholders?
- 10 What are the long-term corrective actions?

5 HINTS FOR ANALYSIS

- Check recent deployment changes.
- Compare latency, error rate and database connection metrics.
- Review application logs and traces across services.
- Identify whether retries are causing additional load.
- Look for connection pool exhaustion.
- Check downstream service health and dependency errors.
- Validate if the issue is isolated to checkout or impacting other flows.
- Examine configuration changes, feature flags and schema changes.

6 POSSIBLE HYPOTHESES

- 1 Database connection pool exhaustion.
- 2 Slow or inefficient database queries.
- 3 Deployment introduced a performance regression.
- 4 Increased retries amplifying load on downstream services.
- 5 Lock contention or resource saturation in the database.
- 6 A downstream dependency is failing and causing cascading delays.
- 7 Configuration change (timeouts, connection limits, thread pools).
- 8 Network, DNS or service mesh issues between services.

7 SAFEST IMMEDIATE MITIGATIONS

-  Reduce retry attempts and timeout values.
-  Increase database connection pool and capacity (if safe).
-  Shift or limit traffic (rate limiting or load shedding).
-  Disable non-essential features.
-  Revert recent configuration changes.
-  Consider rollback (after assessing risks).

8 ROLLBACK CONSIDERATIONS

- Will rollback restore database schema compatibility?
- Are there data migrations that cannot be reversed?
- Will rollback cause data inconsistency or not work with current state?
- Are there other changes in the environment?
- Is a forward fix safer and faster than rollback?

9 RECOVERY VERIFICATION SIGNALS

- ✓ Customer requests succeed.
- ✓ Error rate returns to normal levels.
- ✓ Latency is back within target range.
- ✓ Database connections are stable.
- ✓ Queue backlogs are cleared.
- ✓ All downstream services are healthy.
- ✓ Autoscaling is functioning normally.
- ✓ Synthetic tests pass.
- ✓ No hidden secondary failures.

10 COMMUNICATION TO STAKEHOLDERS

-  Internal (engineering): clear technical updates with impact, mitigation and next steps.
-  Executive: business impact, customer impact, current status, expected resolution time.
-  Customer-facing: concise and transparent updates with known facts and next update time.
-  Avoid speculation. Communicate only verified information.

11 LONG-TERM CORRECTIVE ACTIONS

- ☐ Optimize slow database queries.
- ☐ Improve connection pooling and limits.
- ☐ Add better autoscaling and capacity headroom.
- ☐ Implement circuit breakers and retry policies.
- ☐ Improve testing and performance validation before deployment.
- ☐ Add monitoring and alerting for early detection.
- ☐ Review architecture and dependency design.
- ☐ Update runbooks and incident playbooks.
- ☐ Conduct blameless post-incident review.

12 COMPLETE INCIDENT OPERATING MODEL

- | | |
|--|--------------------------------------|
|  Detect | Identify the issue. |
|  Assess | Determine severity and blast radius. |
|  Mobilize | Assign roles and resources. |
|  Observe | Collect evidence and understand. |
|  Hypothesize | Identify likely causes. |
|  Test | Validate hypotheses. |
|  Narrow | Find the root cause. |
|  Mitigate | Reduce immediate harm. |
|  Recover | Restore normal service. |
|  Verify | Confirm full recovery. |
|  Communicate | Keep stakeholders informed. |
|  Learn | Capture lessons. |
|  Prevent | Implement corrective actions. |

13 SENIOR ENGINEER INSIGHT



A postmortem without completed corrective actions is documentation, not risk reduction.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta